Practical Assignment: Statistical Foundations in Information Retrieval

U23AI052 Submission

Topic

Statistical Foundations in Information Retrieval – Zipf's Law and Heap's Law

Objective

To understand and practically analyze **Zipf's Law** and **Heap's Law** using a collection of text documents and observe how word frequency and vocabulary size behave as the amount of text increases.

Problem Statement

A search engine stores a collection of documents containing news articles, web pages, or student documents. Before building an Information Retrieval (IR) system, it is useful to understand the statistical properties of the text collection.

You are given a text dataset containing multiple documents. Your task is to investigate:

1. **Zipf's Law** – the relationship between a word's frequency and its rank.
2. **Heap's Law** – the relationship between the size of the text collection and the number of unique words (vocabulary).

Part A: Practical Implementation of Zipf's Law

Task

Take a collection of text documents and perform the following steps:

1. Convert all text to lowercase.
2. Tokenize the documents into words.
3. Count the frequency of every word.
4. Sort the words according to their frequency.

5. Assign a rank to each word.
6. Display the **Top 20 most frequent words** with their frequency and rank.
7. Plot a graph of **Word Rank vs. Word Frequency**.
8. Use a log-log graph to observe whether the relationship approximately follows Zipf's Law.

Zipf's Law

Zipf's Law states that the frequency of a word is approximately inversely proportional to its rank:

$f(r) \propto 1/r$

or

$f(r) = C/r$

where:

- `f(r)` = frequency of a word
- `r` = rank of the word
- `C` = constant

Example

Rank	Word	Frequency
1	the	520
2	of	310
3	and	285
4	to	260
5	in	245

The most frequent word should have the highest frequency, while the frequency should generally decrease as rank increases.

Part B: Practical Implementation of Heap's Law

Task

Using the same document collection:

1. Start with a small portion of the documents.
2. Count the total number of words.
3. Count the number of unique words.
4. Gradually increase the amount of text.
5. Record the total number of words and vocabulary size at each stage.
6. Plot **Total Words vs. Vocabulary Size**.
7. Determine whether the vocabulary growth approximately follows Heap's Law.

Heap's Law

Heap's Law describes how vocabulary size grows as the amount of text increases:

$V(N) = K \times N^{\beta}$

where:

- $V(N)$ = vocabulary size
- N = total number of words/tokens
- K = constant
- β = parameter usually between approximately 0.4 and 0.6 for many natural-language collections

Example

Total Words (N)	Unique Words (V)
1,000	420
2,000	710
5,000	1,350
10,000	2,250
20,000	3,700

The vocabulary increases as more documents and words are added, but it generally grows more slowly than the total number of tokens.

Practical Questions

Question 1

Write a Python program that calculates the frequency and rank of words in a given text collection and demonstrates Zipf's Law.

Question 2

Write a Python program that calculates vocabulary growth as the number of tokens increases and demonstrates Heap's Law.

Question 3

Create the following graphs:

1. **Rank vs. Word Frequency**
2. **Log(Rank) vs. Log(Frequency)**
3. **Total Number of Words vs. Vocabulary Size**
4. **Log(Total Words) vs. Log(Vocabulary Size)**

Expected Result

After completing the practical, the student should observe that:

- A small number of words occur very frequently.
- Most words occur relatively rarely.
- Word frequency generally decreases as rank increases.
- Vocabulary size increases when more text is added.
- Vocabulary growth is slower than the growth of the total number of tokens.
- The observed results should approximately follow **Zipf's Law** and **Heap's Law**, although real datasets will not follow them perfectly.

Conclusion

Zipf's Law and Heap's Law are important statistical foundations of Information Retrieval. **Zipf's Law** helps explain the distribution of word frequencies in a collection, while **Heap's Law** describes how vocabulary size grows as the collection becomes larger. Understanding these properties helps in designing efficient search engines, indexing systems, and text-processing systems.

```
import os
import re
```

```
from collections import Counter
import matplotlib.pyplot as plt
import numpy as np


folder = "documents"

all_tokens = []

for filename in os.listdir(folder):

    if filename.endswith(".txt"):

        path = os.path.join(folder, filename)

        with open(path, "r", encoding="utf-8") as file:
            text = file.read()

        text = text.lower()
        words = re.findall(r"\b[a-z]+\b", text)
        all_tokens.extend(words)

print("TOTAL COLLECTION")
print("Total words:", len(all_tokens))
print("Unique words:", len(set(all_tokens)))
print("PART A - ZIPF'S LAW")

frequency = Counter(all_tokens)
sorted_words = frequency.most_common()
print("\nTop 20 most frequent words:")
print("-" * 40)
print(f"{'Rank':<10}{'Word':<15}{'Frequency'}")

for rank, (word, count) in enumerate(sorted_words[:20], start=1):

    print(
        f"{rank:<10}"
        f"{word:<15}"
        f"{count}"
    )
```

```
ranks = np.arange(1, len(sorted_words) + 1)

frequencies = np.array(
    [count for word, count in sorted_words]
)

plt.figure()
plt.plot(ranks, frequencies)
plt.xlabel("Word Rank")
plt.ylabel("Word Frequency")
plt.title("Zipf's Law: Rank vs Word Frequency")
plt.grid()
plt.show()

plt.figure()

plt.loglog(
    ranks,
    frequencies,
    marker="."
)

plt.xlabel("Log(Rank)")
plt.ylabel("Log(Frequency)")
plt.title("Zipf's Law: Log-Log Plot")

plt.grid()

plt.show()


print("PART B - HEAP'S LAW")


total_words = len(all_tokens)

word_counts = []
vocabulary_sizes = []
```

```
# Use 10%, 20%, ..., 100%
for percentage in range(10, 101, 10):

    n = int(total_words * percentage / 100)

    # Take first n words
    current_tokens = all_tokens[:n]

    # Total number of words
    N = len(current_tokens)

    # Number of unique words
    V = len(set(current_tokens))

    word_counts.append(N)
    vocabulary_sizes.append(V)

    print(
        f"Words = {N:<8}"
        f"Vocabulary = {V}"
    )

plt.figure()

plt.plot(
    word_counts,
    vocabulary_sizes,
    marker="o"
)

plt.xlabel("Total Number of Words (N)")
plt.ylabel("Vocabulary Size (V)")
plt.title("Heap's Law: Total Words vs Vocabulary Size")

plt.grid()

plt.show()
plt.figure()
```

```
plt.loglog(
    word_counts,
    vocabulary_sizes,
    marker="o"
)

plt.xlabel("Log(Total Words)")
plt.ylabel("Log(Vocabulary Size)")
plt.title("Heap's Law: Log-Log Plot")

plt.grid()

plt.show()

log_N = np.log(word_counts)
log_V = np.log(vocabulary_sizes)

beta, log_K = np.polyfit(
    log_N,
    log_V,
    1
)


K = np.exp(log_K)


print("HEAP'S LAW PARAMETERS")


print(f"K    = {K:.4f}")
print(f"Beta = {beta:.4f}")

print(
    f"\nEstimated Heap's Law:"
    f"\nV(N) = {K:.4f} * N^{beta:.4f}"
)
```

Outputs:

```
(torch_env) D:\Information Retrieval>python tute3.py
TOTAL COLLECTION
Total words: 1205
Unique words: 595
PART A - ZIPF'S LAW

Top 20 most frequent words:
----------------------------------------
Rank      Word            Frequency
1         com             306
2         gov             31
3         org             30
4         edu             27
5         jp              20
6         co              19
7         net             17
8         uk              16
9         google          15
10        the             12
11        ru              9
12        cn              8
13        and             7
14        au              7
15        de              7
16        ne              7
17        of              6
18        force           5
19        pla             5
20        technology      5
```

```
PART B - HEAP'S LAW
Words = 120     Vocabulary = 72
Words = 241     Vocabulary = 144
Words = 361     Vocabulary = 208
Words = 482     Vocabulary = 268
Words = 602     Vocabulary = 326
Words = 723     Vocabulary = 381
Words = 843     Vocabulary = 435
Words = 964     Vocabulary = 494
Words = 1084    Vocabulary = 544
Words = 1205    Vocabulary = 595
HEAP'S LAW PARAMETERS
K    = 0.9710
Beta = 0.9072

Estimated Heap's Law:
V(N) = 0.9710 * N^0.9072
```

Figures: